Projet de génie logiciel

Raider of the lost Pyramid

INFO5 — Polytech Grenoble — 2026-2027
Auteurs : ELOUAZZANI Soufian - BOUTI Chaimae

L'application est un jeu d'exploration. Le joueur incarne un *raider* qui entre dans la pyramide et doit se déplacer de salle en salle, certaines salles étant verrouillées. Il peut parler à des personnages, ramasser des objets, ouvrir les portes et doit atteindre le trésor, une énorme quantité d'or. La fenêtre montre l'image de la salle où le raider évolue : il y réussit des défis pour passer à la salle suivante.

1 Document d'analyse

1.1 Acteurs

Le seul acteur est le joueur. C'est lui qui lance la partie et qui choisit chaque action. Les autres personnages ne sont pas des acteurs : ils ne prennent aucune initiative. Ils font partie du système et réagissent quand le joueur leur parle ou tente de passer.

1.2 Diagramme de cas d'utilisation

Les cas suivent les actions demandées par le sujet : se déplacer, observer, ramasser, transporter (le sac), utiliser, rencontrer un personnage et lui parler, et relever un défi. Quitter ferme la fenêtre sans sauvegarder l'état de la partie. Trouver le trésor n'est pas un bouton : c'est le résultat d'un déplacement vers la salle du trésor, après avoir réussi le défi qui en garde l'accès.

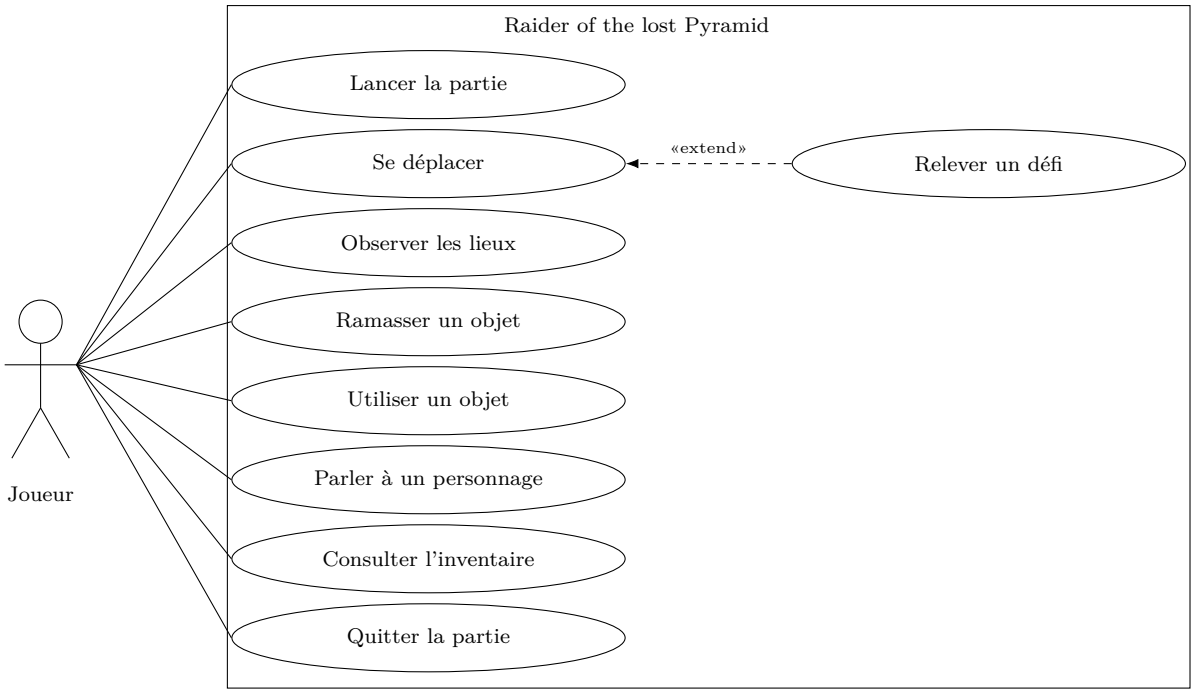


Figure 1 – Diagramme de cas d'utilisation.

- **Lancer la partie.** Le joueur démarre le jeu. Le raider apparaît à l'entrée de la pyramide avec un sac vide et trois vies. La fenêtre affiche l'image de la première salle, sa description et les actions possibles.
- **Se déplacer.** Le joueur choisit une direction proposée pour la salle courante. Scénario nominal : la porte n'est pas verrouillée, et le joueur relève le défi de la salle. Il entre alors dans la salle suivante et la fenêtre change d'image et de texte. Alternatives : la porte est verrouillée et il faut d'abord trouver la clé ou le mécanisme qui l'ouvre ; le défi est échoué (voir ci-dessous).
- **Relever un défi.** Pour franchir une sortie, le joueur doit réussir l'épreuve qui la garde : une énigme par exemple. Scénario nominal : le joueur réussit, le défi est marqué comme résolu, un message le félicite et le passage s'ouvre. Un défi résolu n'est jamais rejoué. Alternatives : mauvaise réponse ou mauvais objet, le joueur perd une vie et peut retenter ; il manque l'objet requis (par exemple la torche), le jeu ne retire pas de vie ; il n'a plus de vie, la partie est perdue.
- **Ramasser un objet.** Le joueur choisit un objet posé dans la salle (torche, corde, clé dorée, amulette, pièces d'or). Scénario nominal : l'objet est transportable et le sac contient encore d'espace (5 éléments maximum). L'objet quitte la salle et entre dans le sac. Alternatives : l'objet n'est pas transportable ; le sac est rempli, le joueur doit libérer d'espace .
- **Utiliser un objet.** Le joueur choisit un objet de son sac. Les utilisations utiles dépendent du lieu : la clé dorée ouvre une porte verrouillée, la torche éclaire une salle sombre, la corde permet de franchir une fosse, l'amulette protège d'un piège.
- **Parler.** Le joueur parle au personnage présent dans la salle. L'archéologue blessé donne un indice sur le prochain défi. Le gardien momifié explique la règle de l'épreuve mais ne cède pas le passage.
- **Observer.** Le joueur relit la description de la salle, ce qui est au sol, les personnages présents et les sorties. En examinant les murs, il peut découvrir une inscription qui aide à résoudre le défi. Dans une salle sombre sans torche, la description reste très limitée.
- **Consulter l'inventaire.** Le jeu écrit dans le message le contenu du sac, le nombre de places restants dans le sac et le nombre de vies restantes. Si le sac est vide, le jeu le dit.

1.3 Diagramme de séquence système

Le diagramme détaille un seul cas, « Se déplacer », avec le cas inclus « Relever un défi ». Le joueur choisit une direction, le système lui propose l'énigme qui garde la sortie, puis le joueur répond. Le diagramme reste au niveau du système : on ne voit pas encore les classes. Les flèches en pointillés sont les réponses du système.

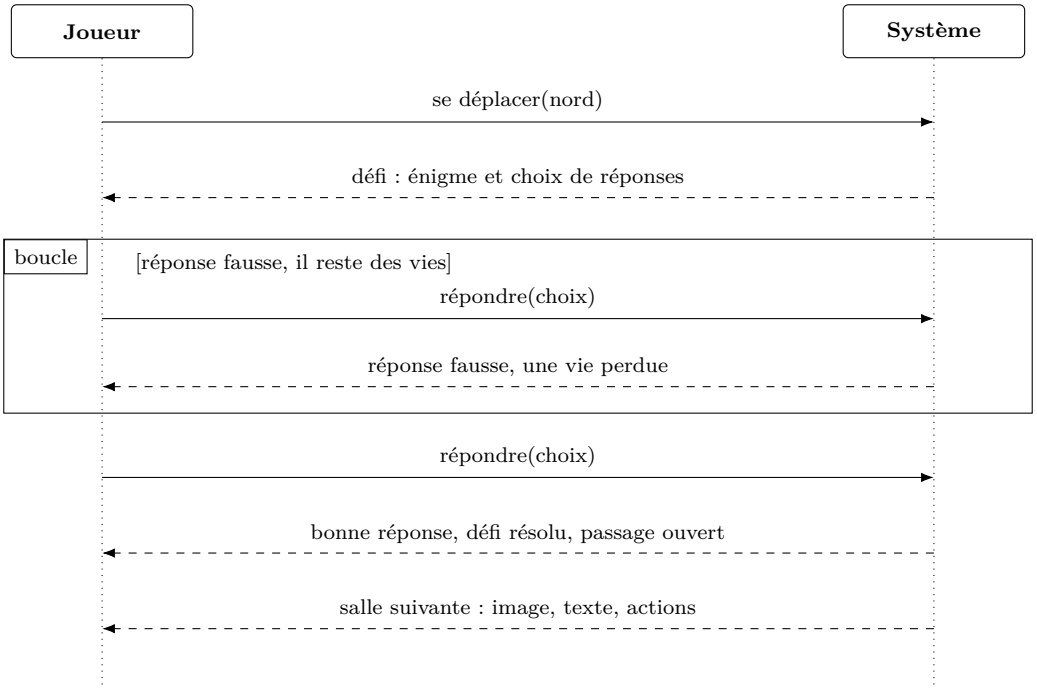


Figure 2 – Séquence système du cas « Se déplacer », avec le cas inclus « Relever un défi ».

Deux alternatives importantes : si le joueur n'a plus de vie après une mauvaise réponse, le système termine la partie sur une défaite, le *raider* est mort et le joueur doit relancer le jeu depuis le début ; si la porte est verrouillée, le système refuse le déplacement tant que la clé n'a pas été utilisée.

1.4 Diagramme de classes d'analyse

Le monde est un graphe de salles. Une partie crée un raider et lance le jeu. Chaque salle a un nom, une description et une image pour présenter l'environnement ; elle indique si elle est verrouillée et si elle contient le trésor(la dernière salle). Une salle verrouillée est ouverte par un objet précis. Une salle mène à d'autres salles, peut contenir des objets transportables, un personnage et des défis. Un défi est une énigme : il a un énoncé, des choix de réponses, une bonne réponse, et mémorise s'il est résolu. Le raider se trouve dans une salle à la fois ; il a un nombre de vies (trois au départ) et un sac de cinq objets au plus. Un personnage a un nom et un dialogue, et peut offrir un objet. La partie mémorise si elle est terminée et gagnée. Les noms du diagramme sont ceux du domaine. Les classes du code, en anglais, sont données dans la conception.

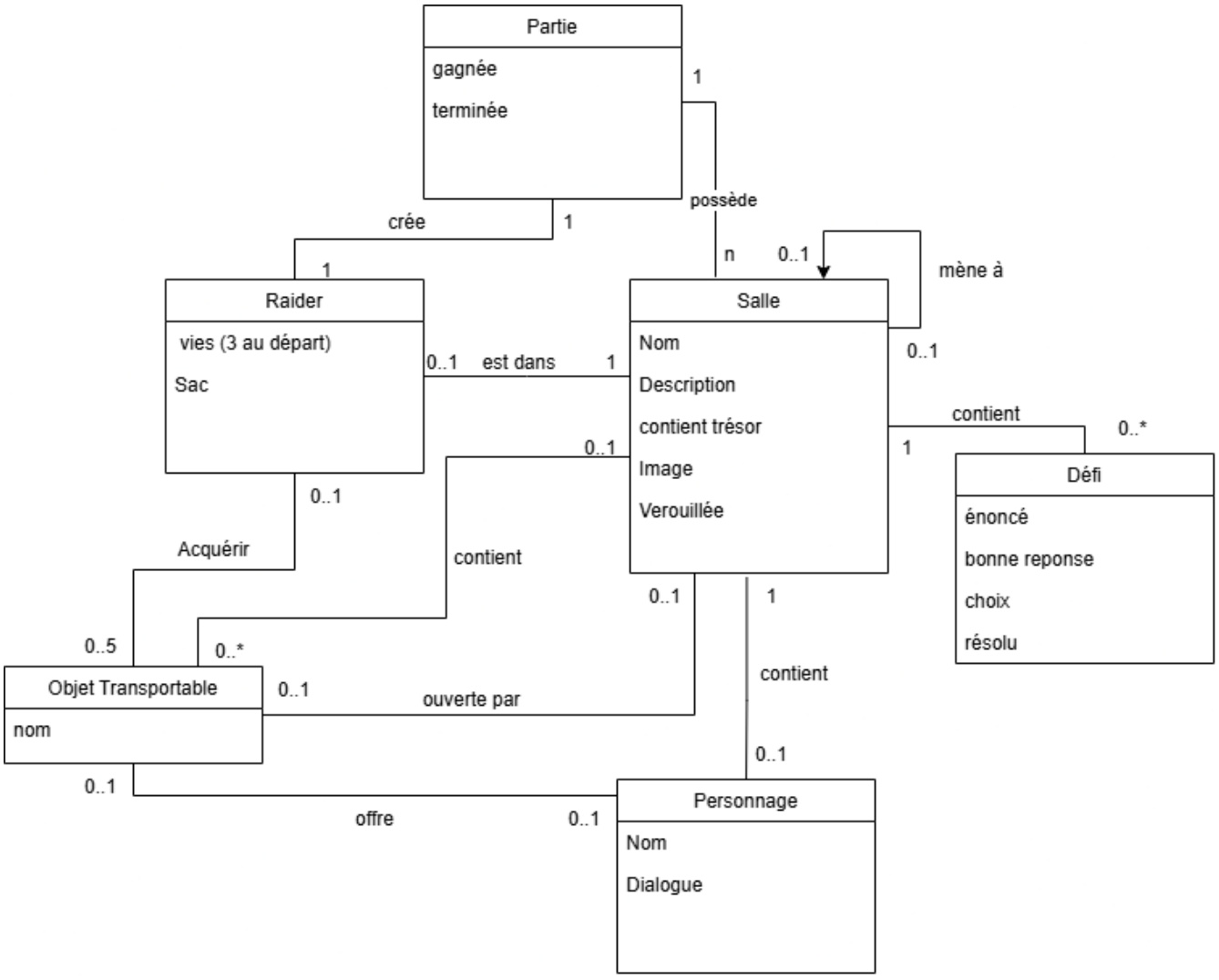


Figure 3 – Diagramme de classe d'analyse

2 Document de conception

2.1 Architecture MVC

Le modèle contient les règles et ne fait aucun affichage. Il n'importe ni Swing ni le contrôleur. La vue est la fenêtre Swing. Le contrôleur est le seul objet qui modifie le modèle puis demande à la vue de se redessiner.

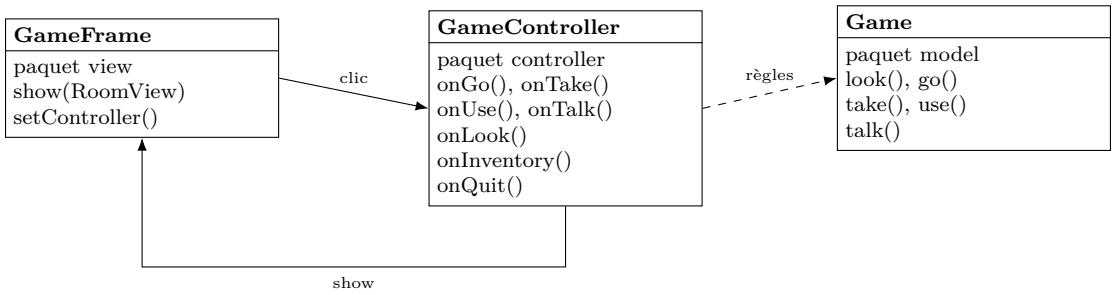
Modèle : `Item`, `Exit`, `Personnage`, `Room`, `Player`, `Game`, `RoomView`. `RoomView` est une copie des informations à afficher (nom, image, sorties, objets, sac, message, partie terminée). La fenêtre ne parcourt pas les salles elle-même.

Vue : `GameFrame`. Elle affiche l'image, le texte (salle, description, personnage, objets, liste des actions possibles), le message, le sac, puis les boutons. Elle ne décide pas si une porte s'ouvre.

Contrôleur : `GameController`. Chaque bouton appelle une méthode (`onGo`, `onTake`, `onUse`, `onTalk`, `onLook`, `onInventory`, `onQuit`). Cette méthode appelle `Game`, puis `show` avec le résultat de `look()`. `Main` crée la partie, la fenêtre et le contrôleur, puis appelle `start()`.

La vue connaît le contrôleur, parce que les boutons l'appellent. Le contrôleur connaît la vue et le

modèle. Le modèle ne connaît ni l'un ni l'autre. On peut donc changer la fenêtre sans réécrire les règles. Les tests de `Game` (`mvn test`) s'exécutent d'ailleurs sans ouvrir de fenêtre.



GameFrame appelle le contrôleur. Le contrôleur appelle Game, puis GameFrame.show. Game ne voit pas la fenêtre.

Figure 4 – Classes de l'architecture et le sens des appels.

2.2 Diagramme de classes logicielles

La classe s'appelle `Personnage` et non `Character`, pour ne pas la confondre avec `java.lang.Character`. `Game` ne garde pas de liste de salles : `createWorld` relie les salles entre elles, et le joueur retient la salle courante. Le reste du graphe reste en mémoire parce que chaque sortie pointe vers une salle.

Carte construite dans `Game.createWorld` : Entrée (nord) Grand hall ; Grand hall (ouest) Ermitage, (est) Couloir, (sud) Entrée ; Couloir (ouest) Grand hall, (est) Salle du trésor. Cette dernière sortie est gardée et verrouillée par la clé rouillée. La clé est au sol dans le hall. L'épée est le cadeau de l'ermite. Le trésor est dans la salle du but, il n'est pas transportable. Entrer dans cette salle met `finished` et `won` à vrai.

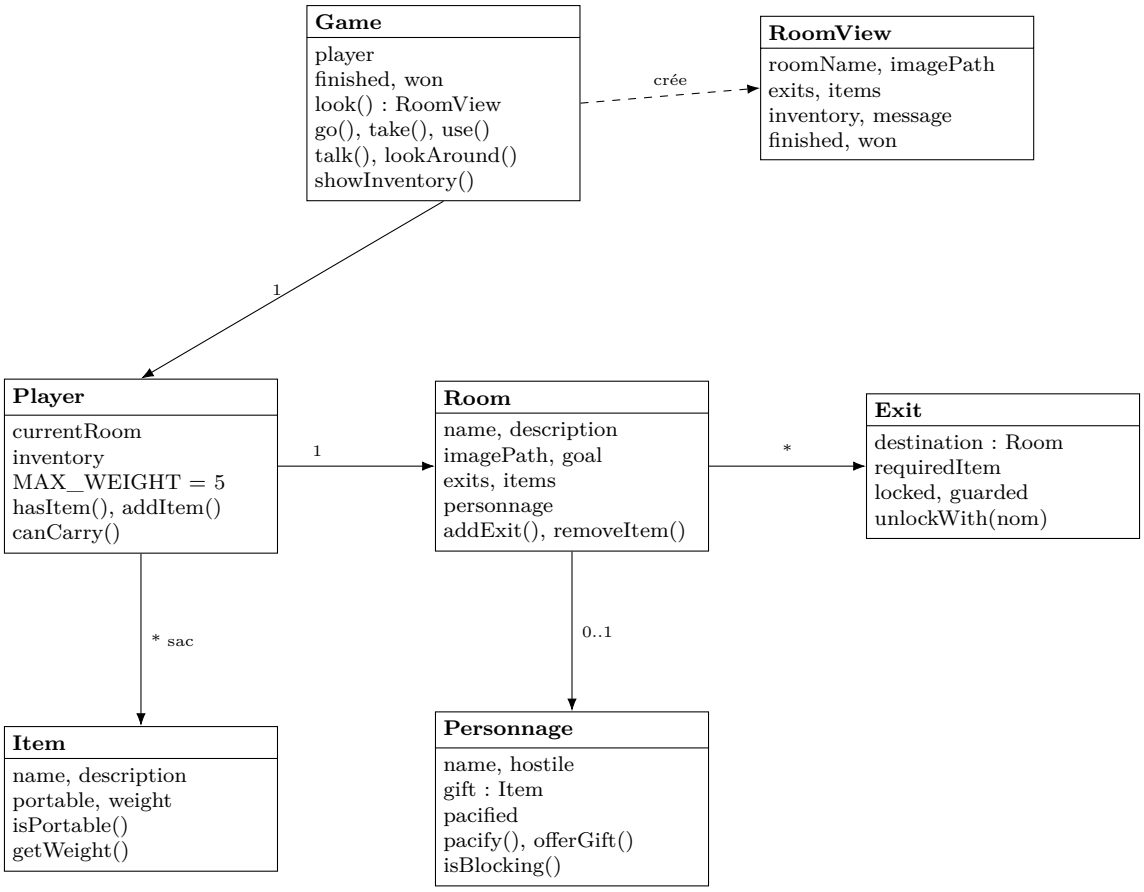


Figure 5 – Classes du modèle. La flèche en pointillés signifie que Game fabrique un RoomView.

2.3 Diagramme de séquence de conception

Le diagramme suit un clic sur « Prendre clé rouillée », depuis le bouton jusqu'au rafraîchissement de la fenêtre. `take` retire l'objet de la salle courante et, s'il est transportable et pas trop lourd, l'ajoute au sac du joueur. `look` construit ensuite le `RoomView` que la vue affiche. Les autres boutons suivent le même chemin : une méthode de `GameController`, une méthode de `Game`, puis `show`.

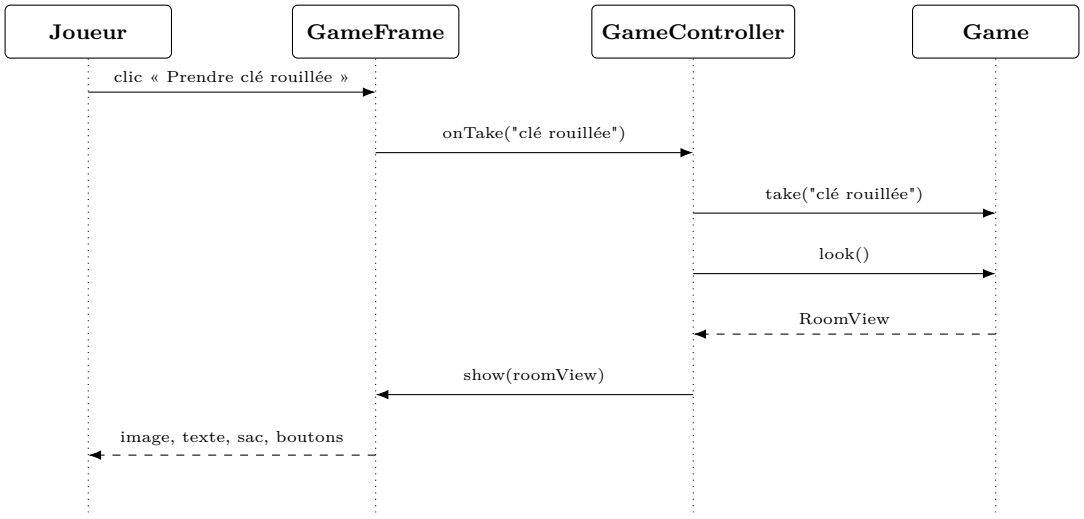
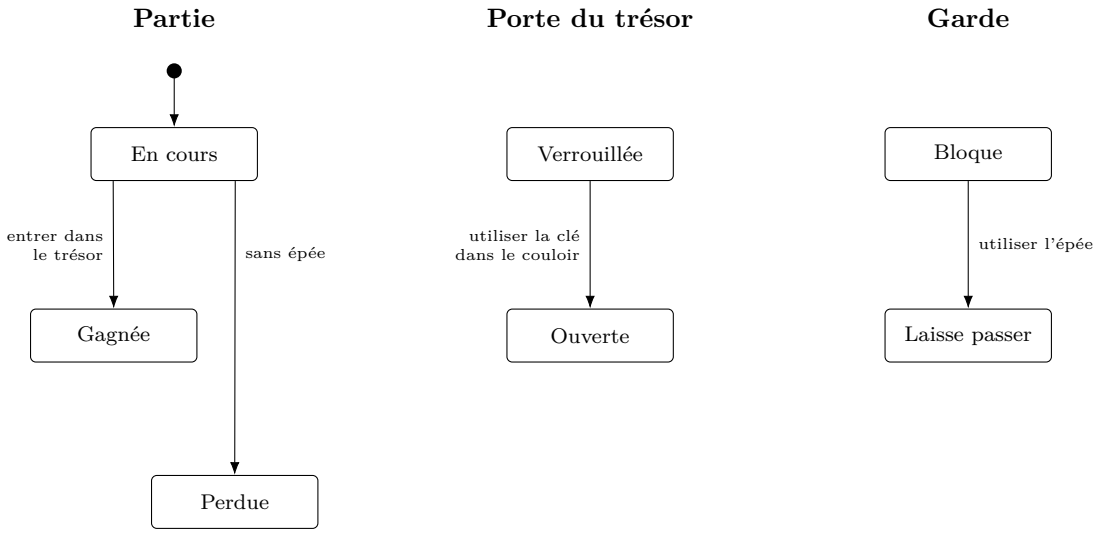


Figure 6 – Séquence MVC pour ramasser la clé. Les retours sont en pointillés.

2.4 Diagrammes d'états-transitions

Trois états méritent un diagramme. La partie commence « en cours ». Elle devient gagnée si le joueur entre dans la salle dont le but est vrai, et perdue s'il force la sortie gardée sans épée. La porte du trésor passe de verrouillée à ouverte quand la clé est utilisée dans le couloir. Le garde passe de « bloque » à « laisse passer » quand l'épée est utilisée dans le couloir. Tant qu'il bloque, une tentative de passage avec l'épée dans le sac ne change pas son état : le joueur doit utiliser l'objet.



Quand la partie est gagnée ou perdue, les boutons d'action sont retirés. Il reste seulement « Quitter ». La porte ne se reverrouille pas.

FIGURE 7 – États de la partie, de la porte et du garde.

3 Manuel utilisateur

3.1 Lancer le jeu

Il faut un JDK 17 (la commande `javac`) et Maven. Dans le dossier qui contient `pom.xml` :

```
mvn -q compile exec:java
```

Une fenêtre « Jeu d'aventures » s'ouvre. Rien à configurer, pas de mot de passe.

3.2 La fenêtre

En haut, l'image de la salle. Au milieu, le nom de la salle, sa description, qui est présent, ce qui est au sol, et la liste « Vous pouvez ». Sous ce texte, un message en gras indique le résultat de la dernière action. En dessous, le sac (poids actuel / 5). Tout en bas, les boutons. Ils reprennent exactement les actions listées dans le texte.

Aller nord, *Aller sud*, *Aller est*, *Aller ouest* : changer de salle quand la direction existe. *Prendre ...* : ramasser l'objet nommé. *Utiliser ...* : utiliser un objet du sac. *Parler* : parler au personnage de la salle. *Observer* : relire la salle. *Inventaire* : rappeler le contenu du sac dans le message. *Quitter* : fermer la fenêtre.

3.3 Carte et partie

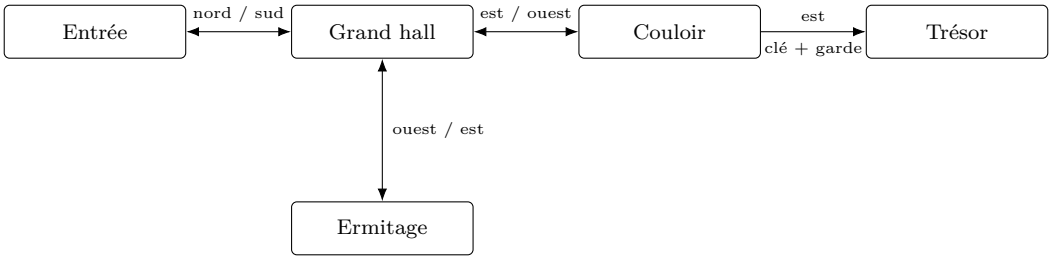


Figure 8 – Carte. La flèche vers le trésor est à sens unique dans la pratique : entrer gagne la partie.

Pour gagner : Aller nord (hall), Prendre clé rouillée, Aller ouest (ermitage), Parler (l'ermite donne l'épée), Aller est puis Aller est (couloir), Utiliser épée, Utiliser clé rouillée, Aller est. Le message « Vous avez trouvé le trésor. Vous avez gagné. » s'affiche et les boutons d'action disparaissent.

Pour perdre : entrer dans le couloir et choisir Aller est sans avoir l'épée. Le garde frappe et la partie s'arrête. Avec l'épée dans le sac, Aller est ne fait que demander d'utiliser l'épée. La clé utilisée hors du couloir ne fait rien. Le trésor s'affiche dans la salle finale mais ne peut pas être emporté.

4 Bilan

Le programme est en Java 17, construit avec Maven, affiché avec Swing (inclus dans le JDK). Les règles sont vérifiées par JUnit, sans fenêtre. Les diagrammes de ce rapport ont été dessinés pour nommer les mêmes classes et les mêmes méthodes que le code.

La difficulté principale était de ne pas mélanger l'affichage et les règles. Un exemple de jeu texte, *World of Zuul* (Barnes et Kölling, *Objects First with Java*), est présent à la racine du dépôt. Il n'a pas été repris : la boucle de jeu et les affichages y sont dans la même classe, et cette version ne compile pas en l'état. Le sujet demande déjà des salles reliées entre elles. Le projet Maven repart de cette idée, avec un modèle qui ne fait que renvoyer un `RoomView`.

L'autre point délicat était la porte gardée et verrouillée : deux conditions sur la même sortie. Elles sont séparées dans `Exit` (`locked`, `guarded`) et décidées dans `Game.go` et `Game.use`, dans un ordre fixe : d'abord le garde, ensuite la serrure. Les états de la figure 7 correspondent à cet ordre.

Le jeu reste volontairement court : cinq salles, deux objets utiles, deux personnages, une victoire et une défaite. Cela suffit pour couvrir le sujet et pour que les diagrammes restent lisibles sur une page A4.